UNIVERSIDAD CATÓLICA BOLIVIANA "SAN PABLO" · ARQUITECTURA DE COMPUTADORAS · EVALUACIÓN PARCIAL 2

Memoria Arquitectónica: Simulador de CPU de 8 Bits, Jerarquía de Memoria Multinivel y Paginación

Equipo de Desarrollo: Lacanasso | **Integrantes:** David Rodrigo Torrico Vera (`@DavidTorricoVe`) y Jorge Aguilera (`@jorgeagch`) | **Repositorio:** `github.com/DavidTorricoVe/lacanasso-simulador-cpu-p2`

1. PROPUESTA DE VALOR Y FILOSOFÍA DE DISEÑO

El presente proyecto evoluciona la arquitectura base del Parcial 1 hacia un computador completo de 8 bits con direccionamiento virtual extendido de 9 bits, jerarquía de caché en dos niveles, paginación bajo demanda con área de intercambio (*Swap*) y periféricos mapeados en memoria. A diferencia de un simulador estático donde las instrucciones se codifican mediante condicionales fijos, nuestra implementación se rige por tres principios de ingeniería de hardware:

- **Ejecución 100% Conducida por Tablas (Cero Hardcoding):** El motor de decodificación no contiene ningún código de operación fijo en su lógica. Toda instrucción, micro-operación de la ALU, registro origen/destino y condición de salto se consulta dinámicamente desde la hoja `ISA`. Si el docente modifica el código hexadecimal de una instrucción en la hoja `ISA`, tanto el ensamblador como el procesador adoptan el cambio de inmediato sin alterar el código fuente.
- **Arbitraje Físico de Buses con Lógica Tri-Estado (High-Z):** La transferencia de datos entre registros, memoria y periféricos se modela sobre cuatro buses independientes (Interno, Direcciones, Datos y Control) gobernados por una palabra de control de 18 señales. En cada micro-operación se verifica que exista a lo sumo un buffer emisor activo (`En_ = 1`), manteniendo los siete emisores restantes en alta impedancia (`Hi-Z`) para impedir colisiones eléctricas en el bus.
- **Trazabilidad Visual sin Solapamiento:** El libro de trabajo separa estrictamente la consola operativa (`Hoja 1`), el almacenamiento secundario de 512 Bytes (`Memoria_Virtual_Swap`), el diccionario del repertorio de instrucciones (`ISA`) y el entorno de desarrollo con ensamblador simbólico de dos pasadas (`Editor`).

2. ESPECIFICACIÓN TÉCNICA DE LA ARQUITECTURA IMPLEMENTADA

Subsistema de Hardware	Especificación Arquitectónica	Ubicación en Interfaz y Módulo Responsable
Núcleo CPU y Registros	4 registros de propósito general de 8 bits (`AX` , `BX` , `CX` , `DX`), registros de sistema `PC` (9 bits), `IR` (8 bits), `MAR` (9 bits), `MDR` (8 bits) y banderas `ZF` , `CF` , `SF` .	`Hoja 1` (`T5:U18`) Módulo `01_Config_Y_Buses.gs` (David Torrico)

Subsistema de Hardware	Especificación Arquitectónica	Ubicación en Interfaz y Módulo Responsable
ALU de 8 Bits	Operaciones `ADD`, `SUB`, `INC`, `DEC`, `CMP`, `AND`, `OR`, `XOR` con monitor binario de 8 bits en tiempo real y actualización automática de banderas de estado.	`Hoja 1` (`W5:AA8`) Módulo `01_Config_Y_Buses.gs` (David Torrico)
Infraestructura de Buses y Unidad de Control	4 buses diferenciados en Fila 4 y generador de micro-palabra de 18 señales (8 buffers Tri-Estado `En_`, 8 latches de carga `Ld_` y 2 líneas de memoria `Mem_R` / `Mem_W`).	`Hoja 1` (`C4:AA4` y `A24:D42`) Módulos `01` y `04` (David Torrico)
Caché Multinivel (L1 Harvard + L2 Unificada)	Nivel 1 separado: `L1I` (1 línea × 8 Bytes para instrucciones) y `L1D` (1 línea × 8 Bytes para datos). Nivel 2 unificado: `L2` (4 líneas × 8 Bytes = 32 Bytes). Políticas *Write-Allocate* y *Write-Through*.	`Hoja 1` (`E24:P41`) Módulo `03_Cache_Jerarquia.gs` (Jorge Aguilera)
Memoria Física (RAM) y Memoria Virtual (Swap)	RAM principal de 256 Bytes dividida en 8 Marcos de Página de 32 Bytes (`Marco 0` a `Marco 7`). Espacio virtual de 512 Bytes dividido en 16 Páginas Virtuales de 32 Bytes (`VPN 00h` a `0Fh`).	`Hoja 1` (`A5:R21`) y hoja `Memoria_Virtual_Swap` Módulo `02_Memoria_Y_Virtual.gs` (Jorge Aguilera)
MMU y Manejador de Fallos de Página	Traducción de direcciones virtuales de 9 bits a físicas de 8 bits. Interrupción por `Bit P = 0` (*Page Fault*), reemplazo configurable `FIFO` o `LRU` sobre los Marcos 1..6 y volcado *Write-Back* si `Dirty = 1`.	`Hoja 1` (`W10:AA13` y `T24:AA42`) Módulo `02_Memoria_Y_Virtual.gs` (Jorge Aguilera)
Periféricos Mapeados en Memoria (Marco 7)	Cola circular FIFO de Teclado de 8 Bytes mapeada en `F0h–F7h` (señal `En_KBD`, instrucción `IN`) y VRAM de 8 Bytes mapeada en `F8h–FFh` (señal `Ld_LCD`, instrucción `OUT`) conectada a pantalla LCD 8×1.	`Hoja 1` (`C21:R21` y `T19:AA23`) Módulo `04_Perifericos_Y_Control.gs` (David Torrico)
IDE Ensamblador de 2 Pasadas y Cargador	Resolución automática de etiquetas simbólicas en Pasada 1, validación sintáctica contra la hoja `ISA` en Pasada 2, generación de código máquina hexadecimal e inyección en memoria.	Hoja `Editor` (`A3:J38`) y menú superior Módulos `04` y `05` (David Torrico & Jorge Aguilera)

3. MODELO MATEMÁTICO DE TRADUCCIÓN MMU E INDEXACIÓN DE CACHÉ

Toda solicitud de memoria generada por el procesador utiliza una dirección virtual de 9 bits (`VAddr` $\in [0, 511]$, expresada como `000h` a `1FFh`). La Unidad de Gestión de Memoria (MMU) y el controlador de caché aplican en tiempo de ejecución las siguientes ecuaciones formales:

1. División de Dirección Virtual en la MMU (Tamaño de Página y Marco = 32 Bytes = 2^5 Bytes):
VPN (Número de Página Virtual) = $\llcorner$ VAddr ÷ 32 $\lrcorner \in [0, 15]$ | Offset (Desplazamiento) = VAddr mod 32 $\in [0, 31]$
Si el bit de presencia cumple **Bit P = 1**, la dirección física de 8 bits se obtiene a partir del Marco Físico asignado (PFN $\in [0, 7]$):
PAddr (Dirección Física en RAM) = (PFN × 32) + Offset $\in [0, 255]$ (`00h` a `FFh`)

2. Indexación de Bloques en Caché L1I, L1D y L2 (Tamaño de Línea = 8 Bytes = 2^3 Bytes):
Tag (Etiqueta de Bloque) = $\llcorner$ VAddr ÷ 8 $\lrcorner$ | Byte Offset = VAddr mod 8 $\in [0, 7]$
Índice de Línea en Caché L2 (4 líneas) = Tag mod 4 $\in \{0, 1, 2, 3\}$ | **Hit Rate (%) = [Hits ÷ (Hits + Misses)] × 100%**

Justificación técnica de protección de marcos en reemplazo `FIFO` / `LRU`: Cuando el procesador referencia una página con `Bit P = 0`, la rutina `HandlePageFault()` selecciona la página víctima exclusivamente entre los **Marcos Físicos 1 al 6**. El `Marco 0` (`00h–1Fh`) se preserva en memoria principal por alojar el segmento de código activo apuntado por `PC`, mientras que el `Marco 7` (`E0h–FFh`) está reservado físicamente para los registros de Entrada/Salida del teclado y la pantalla LCD, los cuales operan como regiones no cacheables (*Uncacheable Memory-Mapped I/O*).

4. CATÁLOGO DE PROGRAMAS DE VALIDACIÓN Y CASOS DE PRUEBA

El simulador incorpora tres algoritmos demostrativos accesibles desde la pestaña `Editor` o desde el menú superior `SIMULADOR LACANASSO`, diseñados para someter a prueba todos los bloques de hardware:

Programa del Catálogo	Subsistemas Evaluados	Traza Técnica y Resultado Verificable en Consola
Programa 1: Cálculo Aritmético Iterativo con Bucles y 4 Registros	Registros `AX, BX, CX, DX`, ALU (`ADD, INC, DEC, SUB`), bandera `ZF` y salto condicional `JNZ`.	Inicializa `AX=00h`, `BX=06h`, `CX=04h`, `DX=02h`. Ejecuta 4 iteraciones sumando `AX + BX` e incrementando `DX` mientras `ZF=0`. Al salir del bucle (`AX=18h`, `DX=06h`), calcula `SUB AX, DX = 12h` (18_{10}) y lo almacena en `80h` (`Marco 4`, activando `Dirty=1`).
Programa 2: Eco Interactivo Teclado FIFO a Pantalla LCD 8×1	Memory-Mapped I/O en `Marco 7`, instrucciones `IN` / `OUT` y señales `En_KBD` / `Ld_LCD`.	Encola los 5 caracteres ASCII de `"HOLA!"` en `F0h–F4h`. Un bucle controlado por `CX=05h` desencola cada byte desde `F0h` con `IN DX, [KBD]` (desplazando la cola FIFO hacia la izquierda) y lo transfiere con `OUT [LCD], DX` hacia la VRAM (`F8h–FCh`), encendiendo `H O L A !` en el display.

Programa del Catálogo	Subsistemas Evaluados	Traza Técnica y Resultado Verificable en Consola
Programa 3: Estrés de Caché Multinivel y Fallos de Página con Write-Back	Caché `L1D/L2` (*Write-Allocate*), interrupción *Page Fault*, reemplazo `FIFO/LRU` y volcado a Swap.	1) `STORE [40h], AX` escribe `AAh` en `VPN 02h` (`Marco 2`), fijando `Dirty=1` y alojando el bloque `Tag 08h` en `L1D` . 2) `LOAD BX, [40h]` y `LOAD CX, [41h]` generan **2 Cache Hits consecutivos en L1D**. 3) `VSTORE [80h], AX` y `VLOAD DX, [A2h]` referencian las páginas ausentes `VPN 08h` y `VPN 0Ah` (`Bit P=0`), disparando **2 Page Faults**, volcando los 32 Bytes sucios del `Marco 2` hacia `Memoria_Virtual_Swap` (*Write-Back*) y cargando el dato `99h` desde el Swap en `DX` .

5. INGENIERÍA DE SOFTWARE, METODOLOGÍA GITFLOW Y CUMPLIMIENTO DE RÚBRICA

El desarrollo se gestionó íntegramente bajo el flujo de trabajo **Gitflow** en el repositorio oficial de GitHub, articulado con un tablero **Kanban** compuesto por **18 Issues técnicos** distribuidos equitativamente entre ambos integrantes:

- **Modularización Limpia en 5 Archivos (`.gs`):** El código fuente se divide en `01_Config_Y_Buses.gs` (configuración centralizada `UI_CONFIG` , ALU y buffers tri-estado), `02_Memoria_Y_Virtual.gs` (RAM en marcos, MMU y manejador de *Page Faults*), `03_Cache_Jerarquia.gs` (caché L1I/L1D/L2 y métricas), `04_Perifericos_Y_Control.gs` (ciclo de instrucción de 4 fases, periféricos de E/S e IDE ensamblador de 2 pasadas) y `05_Constructor_Diseno_UI.gs` (menú superior e inicialización visual).
- **Control de Calidad mediante Pull Requests y Commits Semánticos:** Cada funcionalidad se desarrolló en ramas independientes (`feature/` y `fix/`) con mensajes estandarizados (`feat:` , `fix:` , `refactor:` , `style:` , `docs:`), fusionadas hacia `develop` tras revisión cruzada de código (*Peer Review*) antes del lanzamiento final en `main` , acompañadas de documentación técnica bilingüe (Español e Inglés) con diagramas de arquitectura en formato Mermaid.